sechat

Secure, Real-Time, End-to-End Encrypted Group Chat

FastAPI | WebSockets | AES-256-GCM | PBKDF2 | bcrypt | SQLite

BS Srivallabha (24BCE0457)

Ishaan Singh (24BCE0671)

Computer Networks Project

Project Documentation

October 2026

Contents

1. Aim of the Project
2. Problem Statement
3. Proposed Solution
4. Features
5. Technology Stack
6. System Architecture
7. Workflow & Sequence Diagrams
8. Database Design
9. API Reference
10. Security Model
11. Application Screenshots
12. Conclusion & Future Scope

1. Aim of the Project

> To design and implement **sechat** – a lightweight, browser-based, real-time messaging platform that lets users create password-protected chat rooms and private direct messages, where every message is **encrypted end-to-end on the client** so that the server (and anyone who compromises it) can never read the conversation.

The project also demonstrates core Computer Networks concepts: full-duplex communication over **WebSockets**, the client–server model, the HTTP request/response cycle, token-based connection admission, and application-layer cryptography.

2. Problem Statement

- **Server-side plaintext:** Many chat systems store messages in readable form – a single breach or a curious admin exposes every conversation.
- **Polling is inefficient:** Traditional HTTP polling adds latency, wastes bandwidth and does not scale for live chat.
- **Heavy setup:** Secure messengers usually require mobile apps, phone numbers or complex key exchange.
- **Weak access control:** Ad-hoc group chats often lack password protection and leak to anyone who has the link.

- **Credential storage:** Storing user or room passwords in plain text is a common, dangerous mistake.

3. Proposed Solution

sechat solves these issues with a **zero-knowledge relay** design: the browser derives an AES-256 key from the room password using PBKDF2 (100,000 iterations, room code as salt), encrypts every message with AES-GCM before it ever leaves the device, and the FastAPI server simply authenticates, stores and broadcasts opaque ciphertext over persistent WebSocket connections. Passwords are stored only as bcrypt hashes, and WebSocket connections are admitted only with a one-time token issued after password verification.

4. Features

🔐 User Accounts
Sign-up / sign-in with bcrypt-hashed passwords; unique usernames enforced by the database.

🏠 Password-Protected Rooms
Create rooms with a title, unique code and password; others join using code + password.

🛡️ End-to-End Encryption
AES-256-GCM via Web Crypto API with a random 96-bit IV per message. Server sees only ciphertext.

🔑 Key Derivation
PBKDF2-SHA256, 100k iterations, room code as salt – the key never leaves the browser.

⚡ Real-Time Messaging
Persistent full-duplex WebSockets deliver messages instantly to all room members.

📚 Multi-Room Sessions
One socket per joined room runs in the background; switch between chats instantly.

🔴 Unread Badges
Messages arriving in inactive rooms increment a live unread counter in the sidebar.

👥 Online Members
See who is currently connected to a room through the Members panel.

💬 Direct Messages
Click any member/author to open a private DM room with a deterministic server-derived secret.

🕓 Message History
Last 50 (encrypted) messages are replayed on join and decrypted locally.

🧹 Clear Chat History
Wipe a room's history from the DB; all connected clients are cleared in real time.

🎟️ One-Time Socket Tokens
WebSocket upgrade requires a single-use token, preventing unauthorised connections.

📣 System Notifications
"user joined the group" events broadcast to everyone in the room.

🚪 Leave Room / Sign Out
Close sockets, clear in-memory state and per-user saved rooms cleanly.

🎨 Modern UI
Responsive split-pane layout, coloured usernames, chat bubbles and modals.

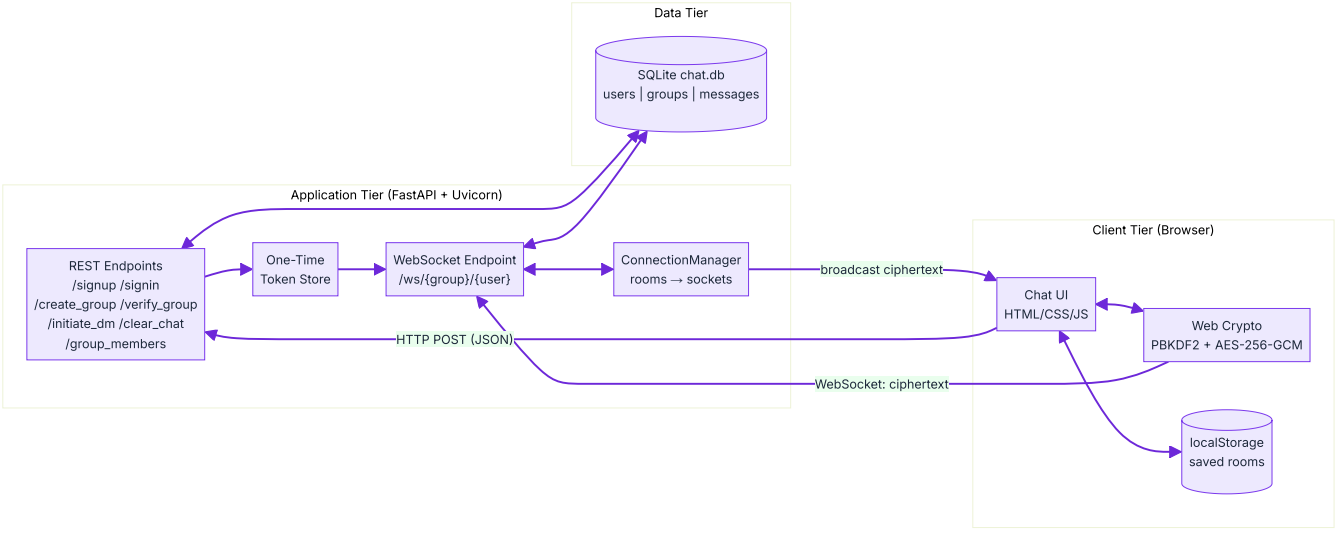
🧱 XSS Protection
All user content is HTML-escaped before rendering.

5. Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, Vanilla JavaScript	Single-page UI served by FastAPI
Cryptography	Web Crypto API (AES-GCM, PBKDF2)	Client-side E2E encryption
Transport	WebSocket (RFC 6455) over TCP, HTTP/1.1	Real-time full-duplex messaging, REST calls
Backend	Python, FastAPI, Uvicorn (ASGI)	REST endpoints, WebSocket server
Connection Mgmt	`ConnectionManager`	Tracks sockets per room, broadcasting
Database	SQLite + aiosqlite	Async storage of users, groups, messages
Password Hashing	bcrypt	Salted hashing of user and room passwords

6. System Architecture

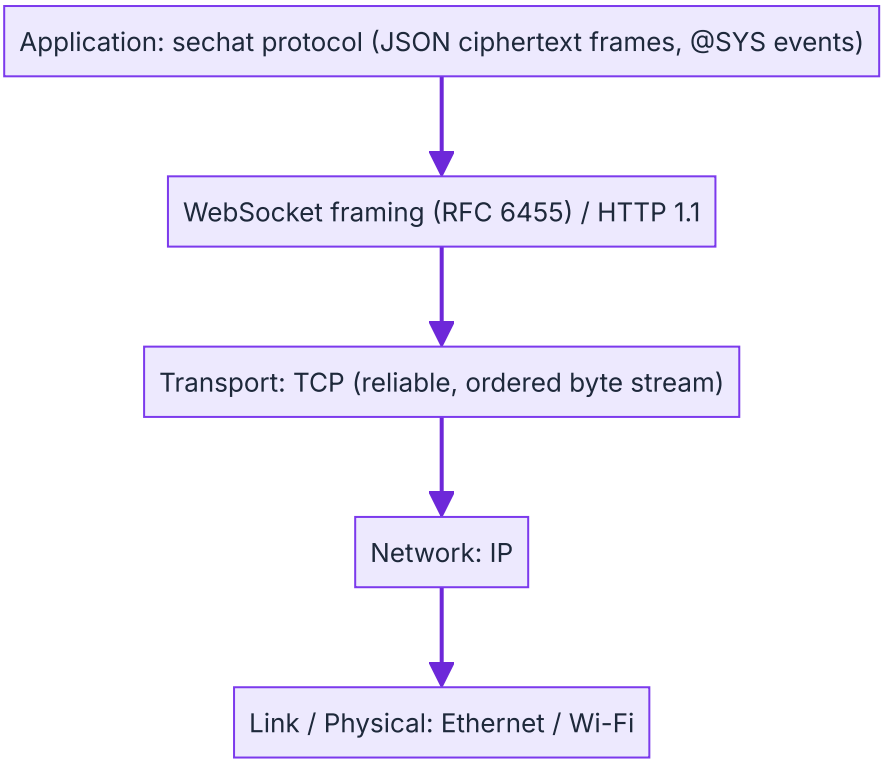
sechat follows a three-tier client–server architecture. Clients talk to the server over HTTP (REST) for authentication and room management, and over WebSockets for live messaging. Encryption and decryption happen only in the client tier.



Component Responsibilities

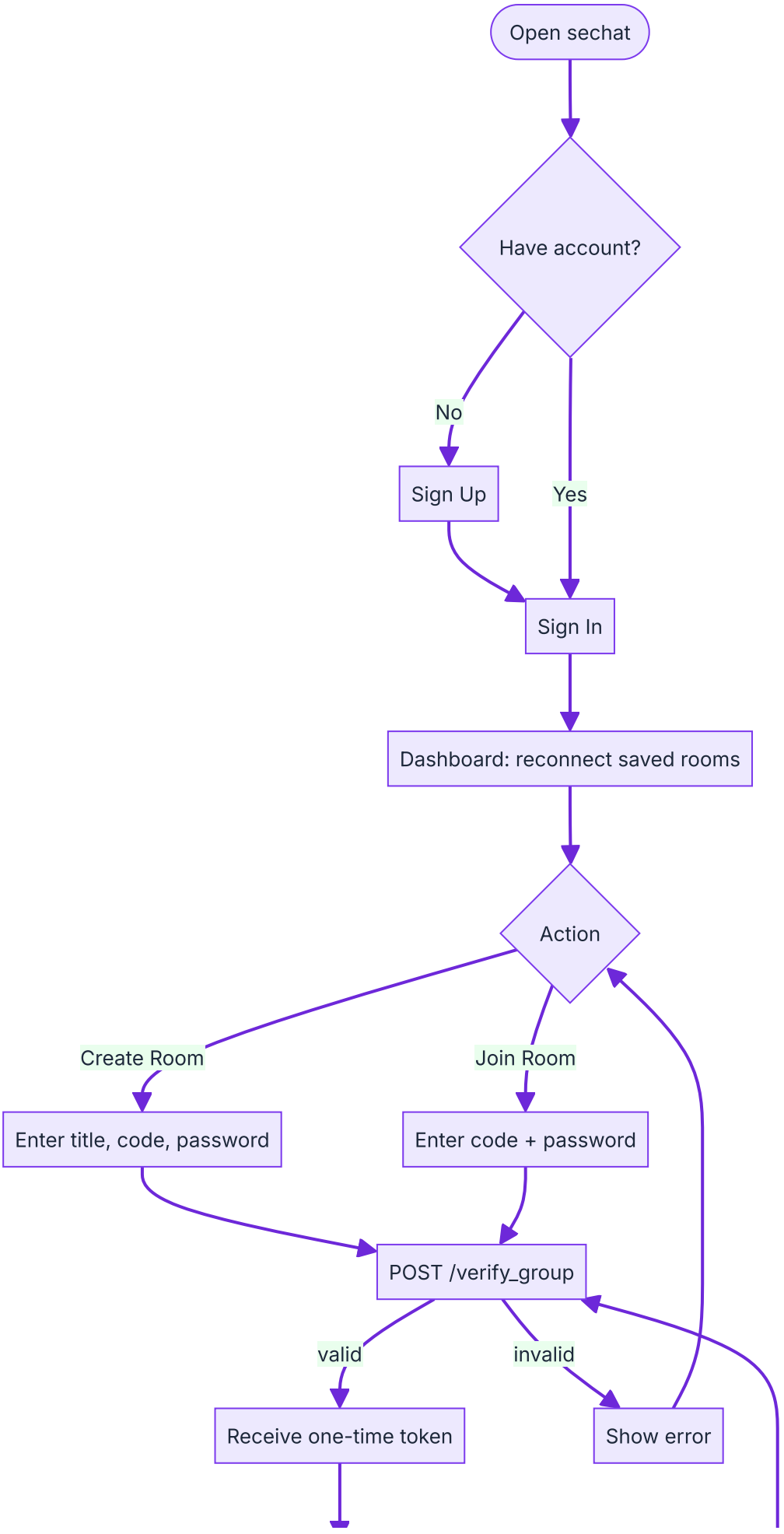
Component	Responsibility
Chat UI (`main.py` HTML)	Auth screens, sidebar, chat window, modals, multi-socket state, unread counters
Web Crypto layer	Derives room key, encrypts outgoing and decrypts incoming messages
REST API (`main.py`)	User and room management, issues one-time WebSocket tokens, DM setup
WebSocket endpoint	Validates token, replays history, persists and relays messages
`ConnectionManager` (`chat_manager.py`)	Maps room → {socket: username}; broadcast & member listing
`database.py`	Schema creation, bcrypt hashing, async CRUD on SQLite

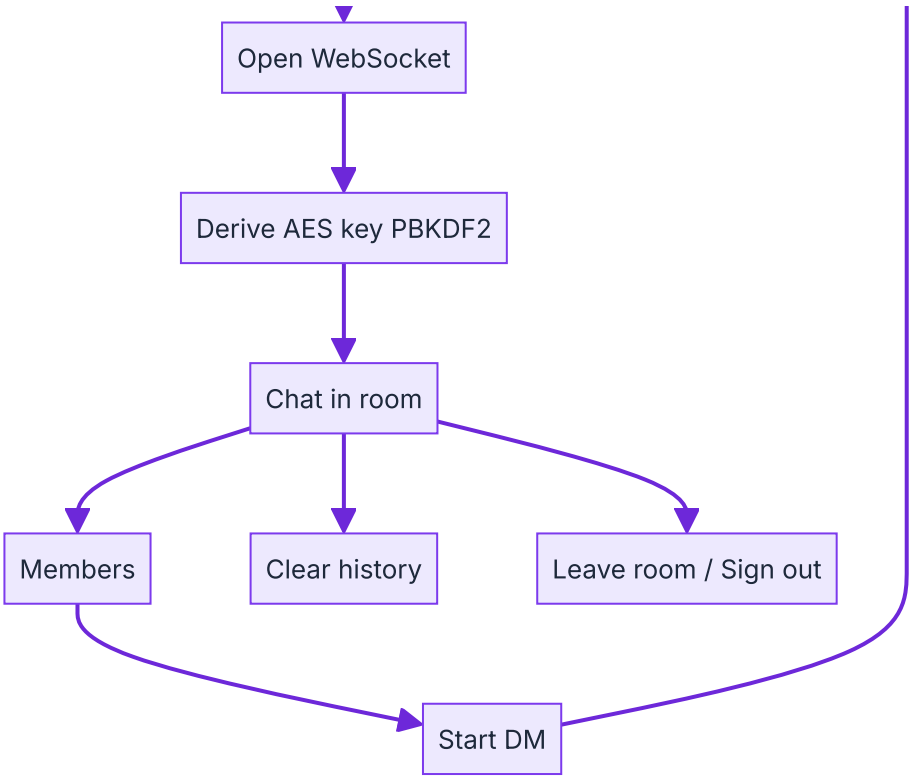
Network Layer View


Application: sechat protocol (JSON ciphertext frames, @SYS events)
WebSocket framing (RFC 6455) / HTTP 1.1
Transport: TCP (reliable, ordered byte stream)
Network: IP
Link / Physical: Ethernet / Wi-Fi

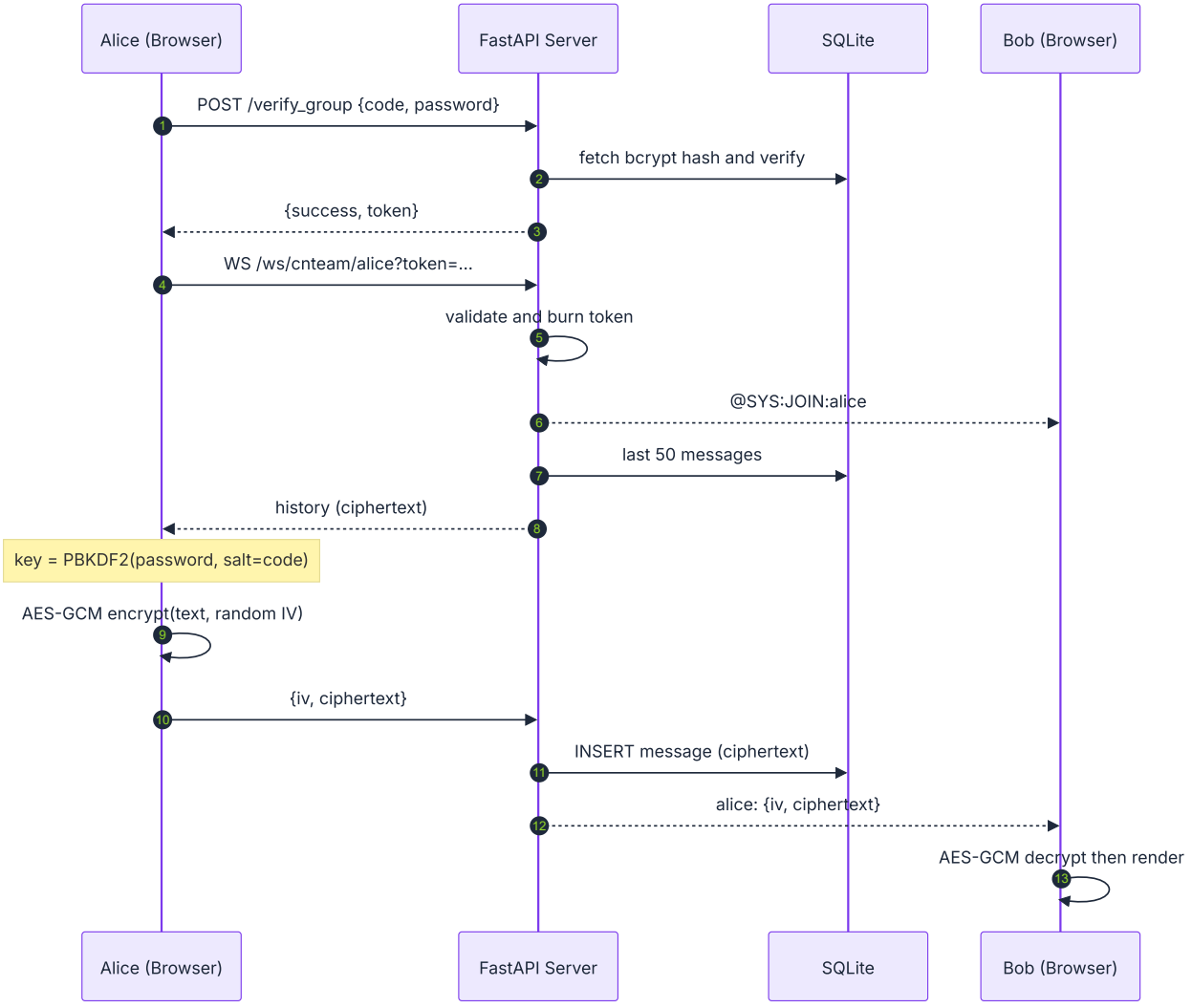
7. Workflow

7.1 Overall User Workflow

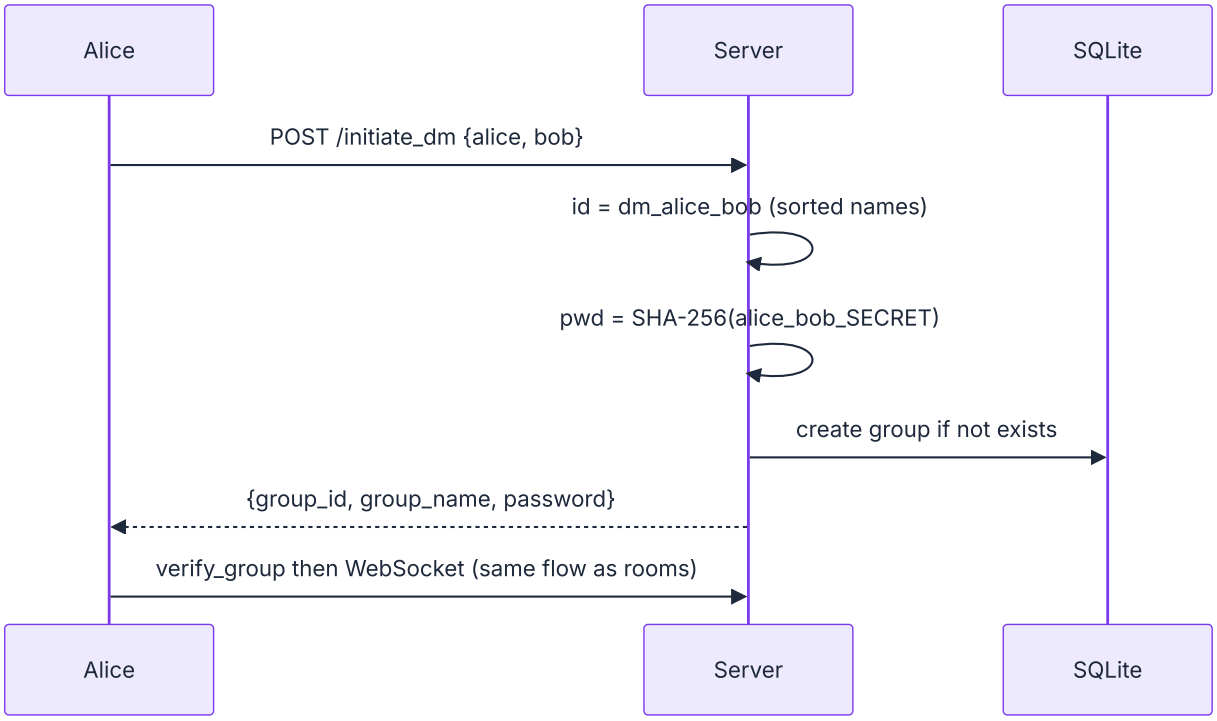

Open sechat
Have account?
No
Yes
Sign Up
Sign In
Dashboard: reconnect saved rooms
Action
Create Room
Join Room
Enter title, code, password
Enter code + password
POST /verify_group
valid
invalid
Receive one-time token
Show error



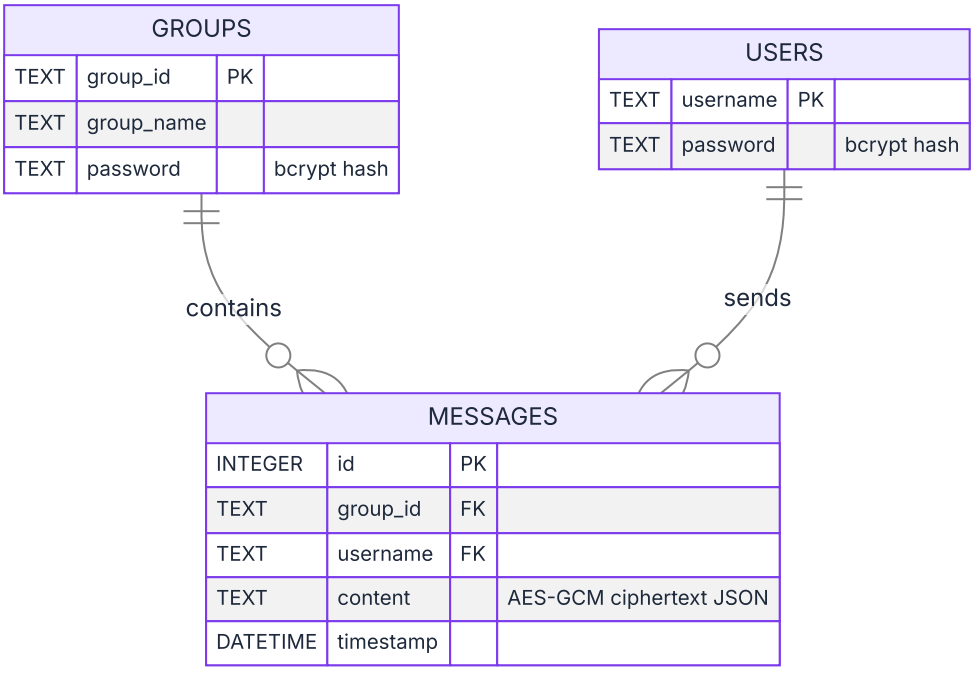
7.2 Encrypted Message Sequence



7.3 Direct Message Setup



8. Database Design



9. API Reference

Method	Endpoint	Description
GET	`/`	Serves the single-page web client
POST	`/signup`	Create user (bcrypt-hashed password)
POST	`/signin`	Verify user credentials
POST	`/create_group`	Create a password-protected room
POST	`/verify_group`	Verify room password, return one-time WS token
GET	`/group_members/{id}`	List users currently connected to a room
POST	`/initiate_dm`	Create/fetch deterministic DM room for two users
POST	`/clear_chat`	Delete room history and broadcast clear event
WS	`/ws/{group_id}/{username}?token=`	Real-time encrypted messaging channel

10. Security Model

Threat	Mitigation in sechat
Database leak / malicious server admin	Messages stored only as AES-256-GCM ciphertext; keys never reach the server
Password theft from DB	User and room passwords stored as salted bcrypt hashes
Unauthorised socket connection	WebSocket requires a single-use token issued only after password verification; invalid → close code 1008
Message tampering	AES-GCM authentication tag – modified ciphertext fails to decrypt
Replay of IV / pattern analysis	Fresh random 12-byte IV for every message
Brute-forcing room key	PBKDF2 with 100,000 SHA-256 iterations slows offline guessing
Cross-site scripting	All usernames and message bodies are HTML-escaped
SQL injection	Parameterised queries throughout `database.py`

> **Encrypted payload format:** `{"iv": "<base64>", "ciphertext": "<base64>"}` – this is exactly what travels over the network and what is stored in the `messages` table.

11. Application Screenshots

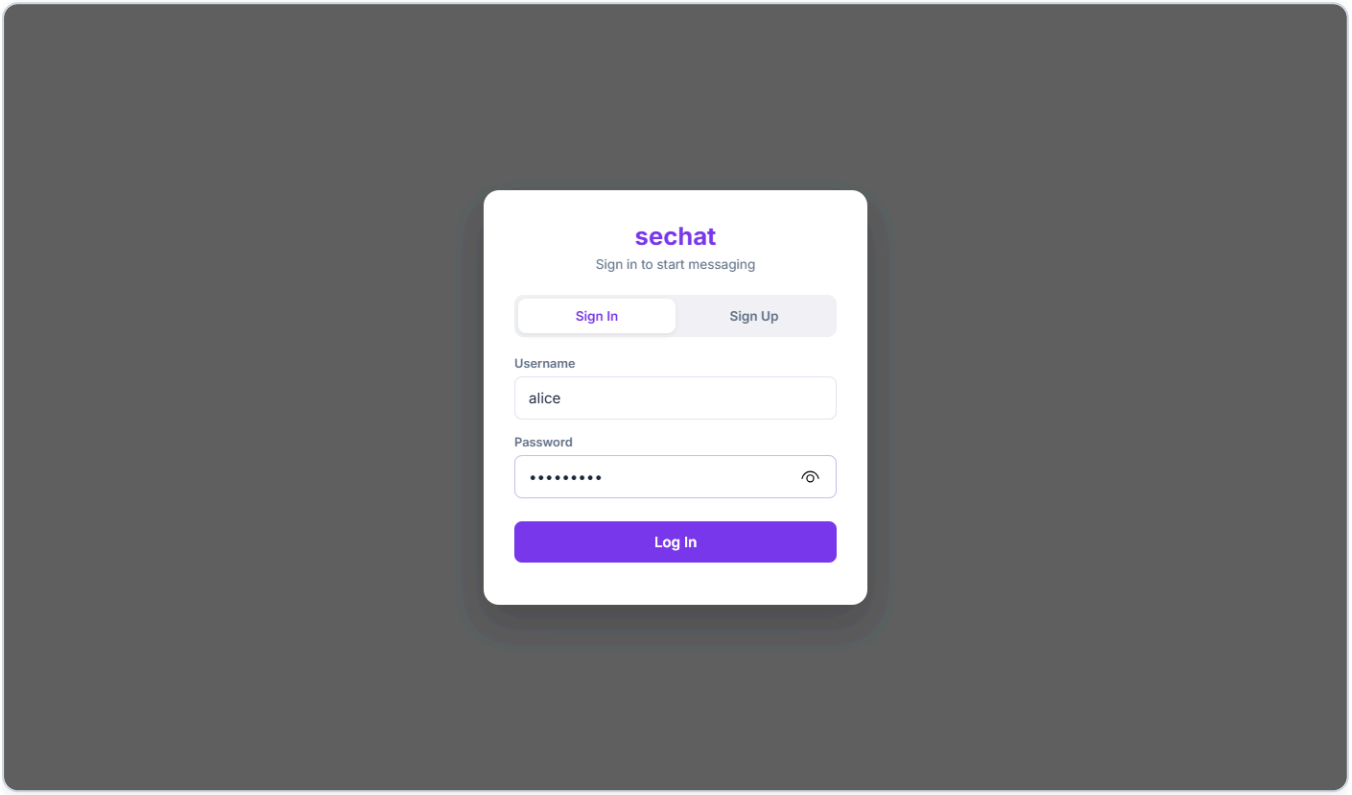


Figure 1 – Sign-in screen

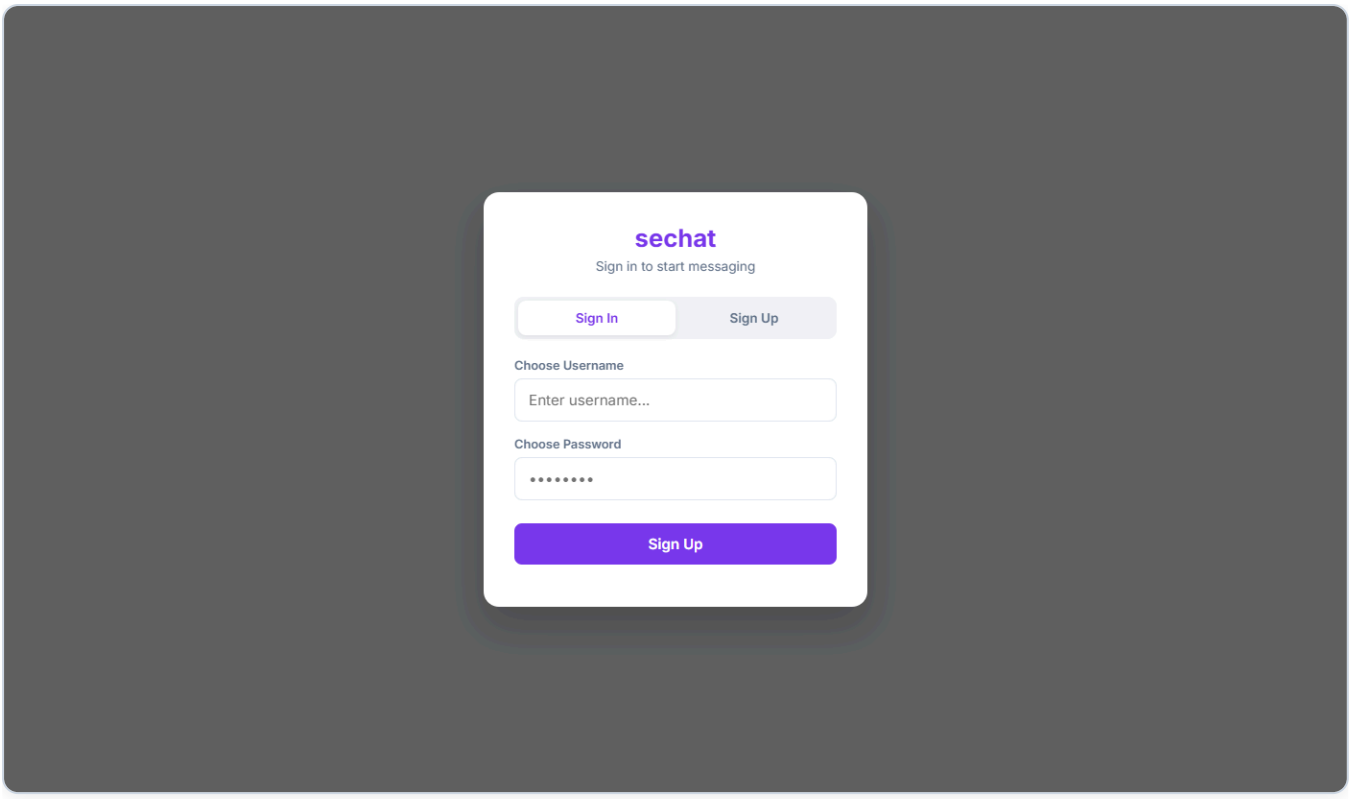


Figure 2 – Sign-up screen

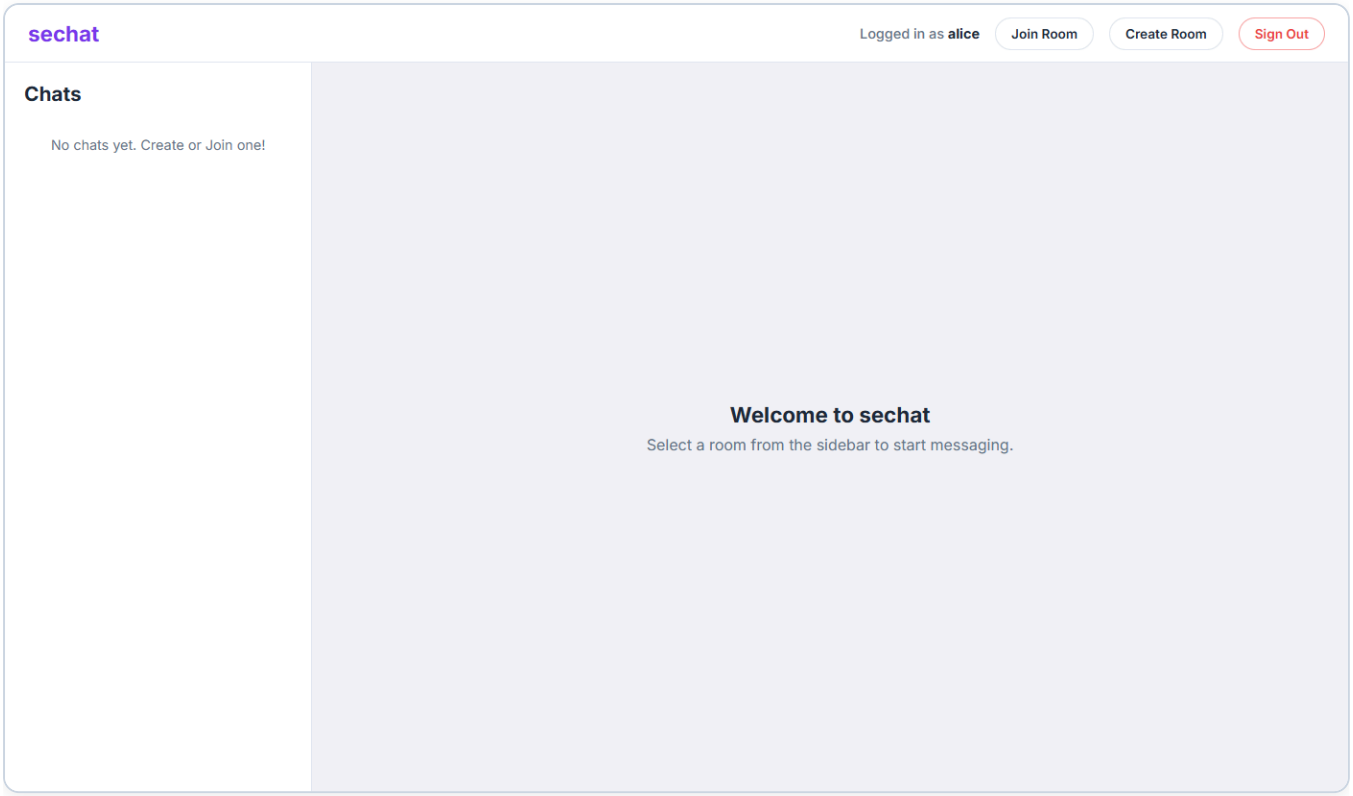


Figure 3 – Dashboard after login (no rooms yet)

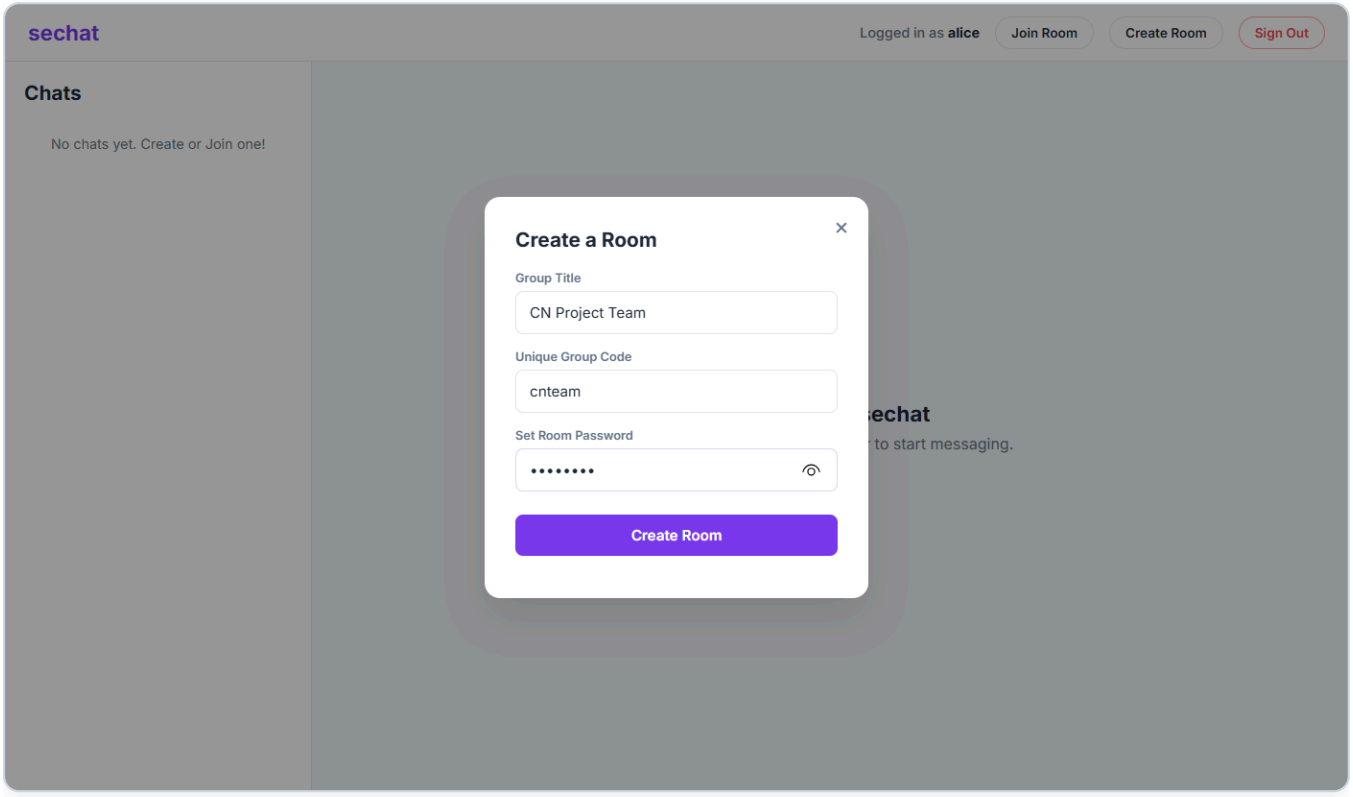


Figure 4 – Creating a password-protected room

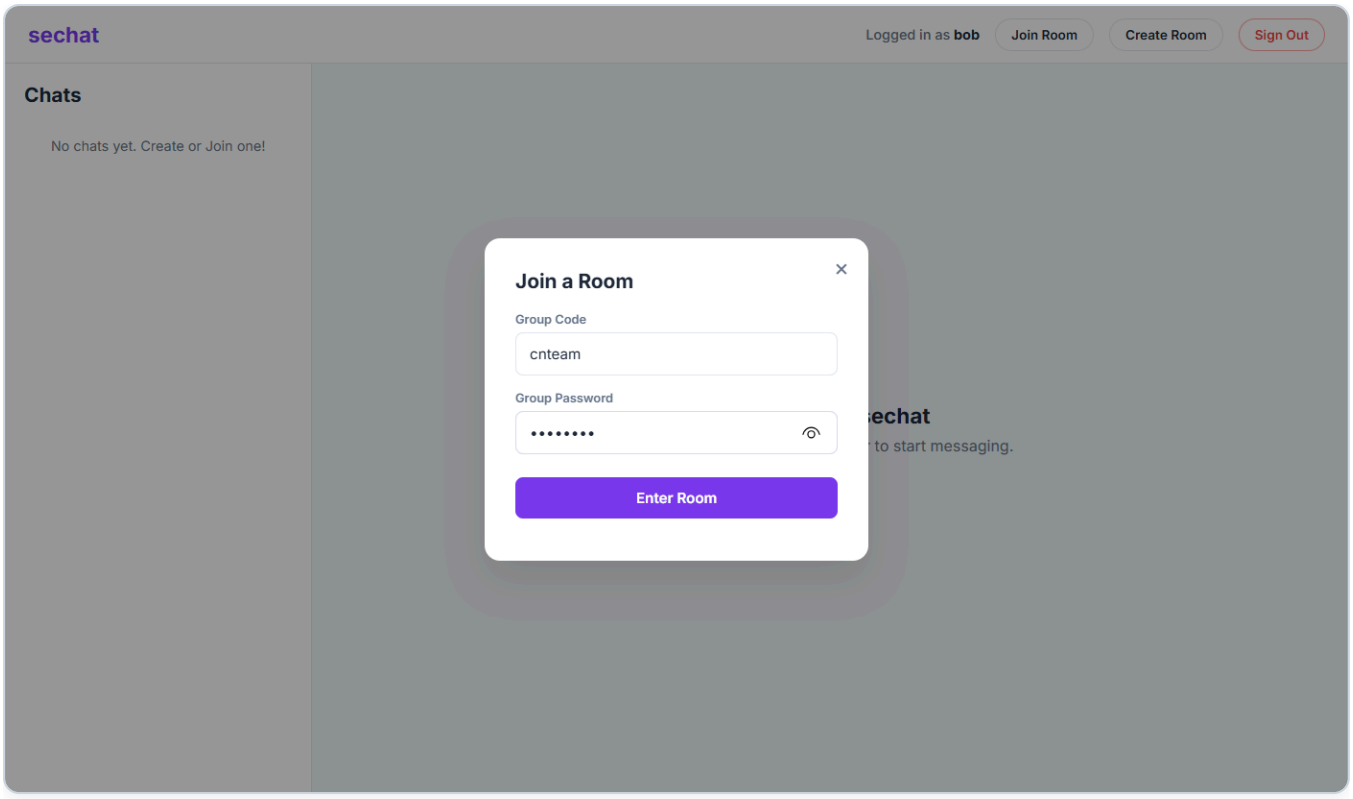


Figure 5 – Joining a room with group code and password

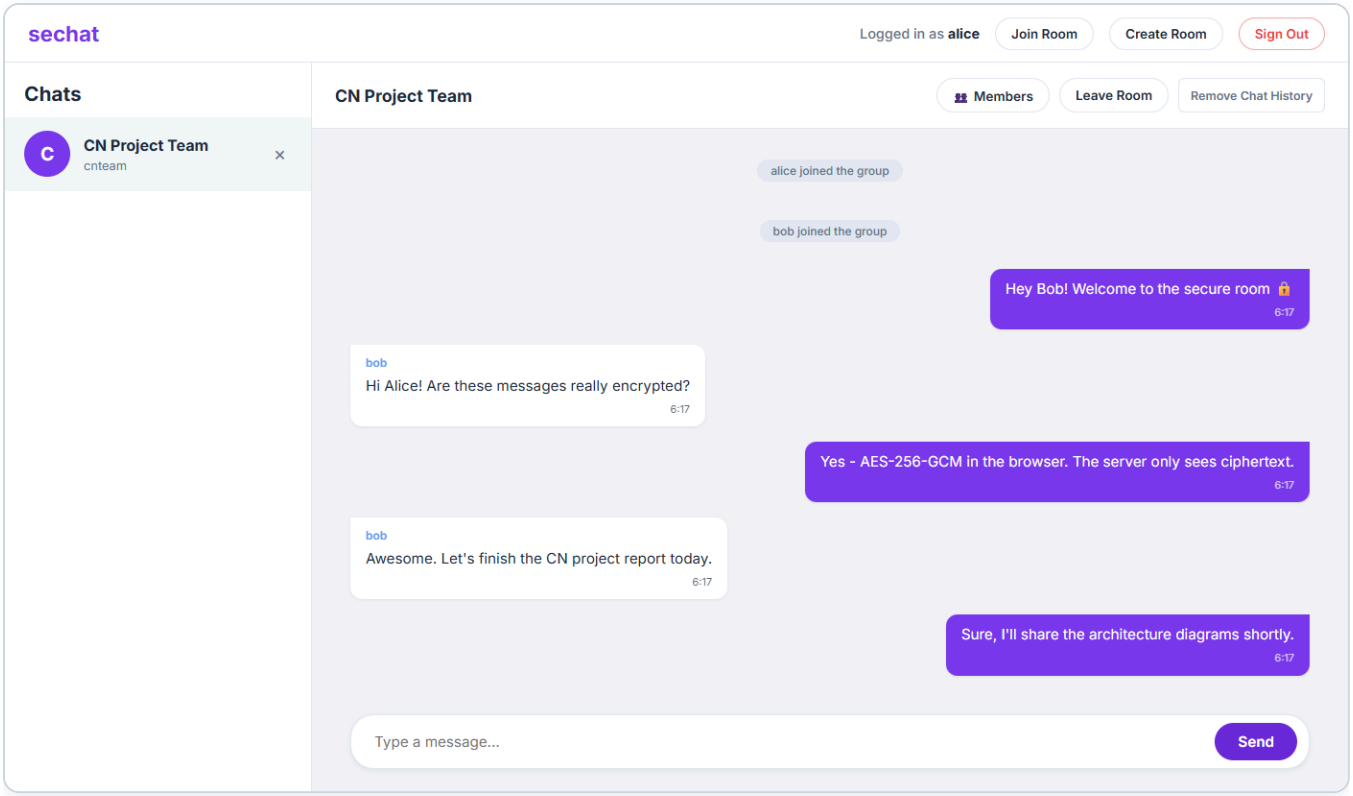


Figure 6 – Real-time encrypted group chat (Alice's view)

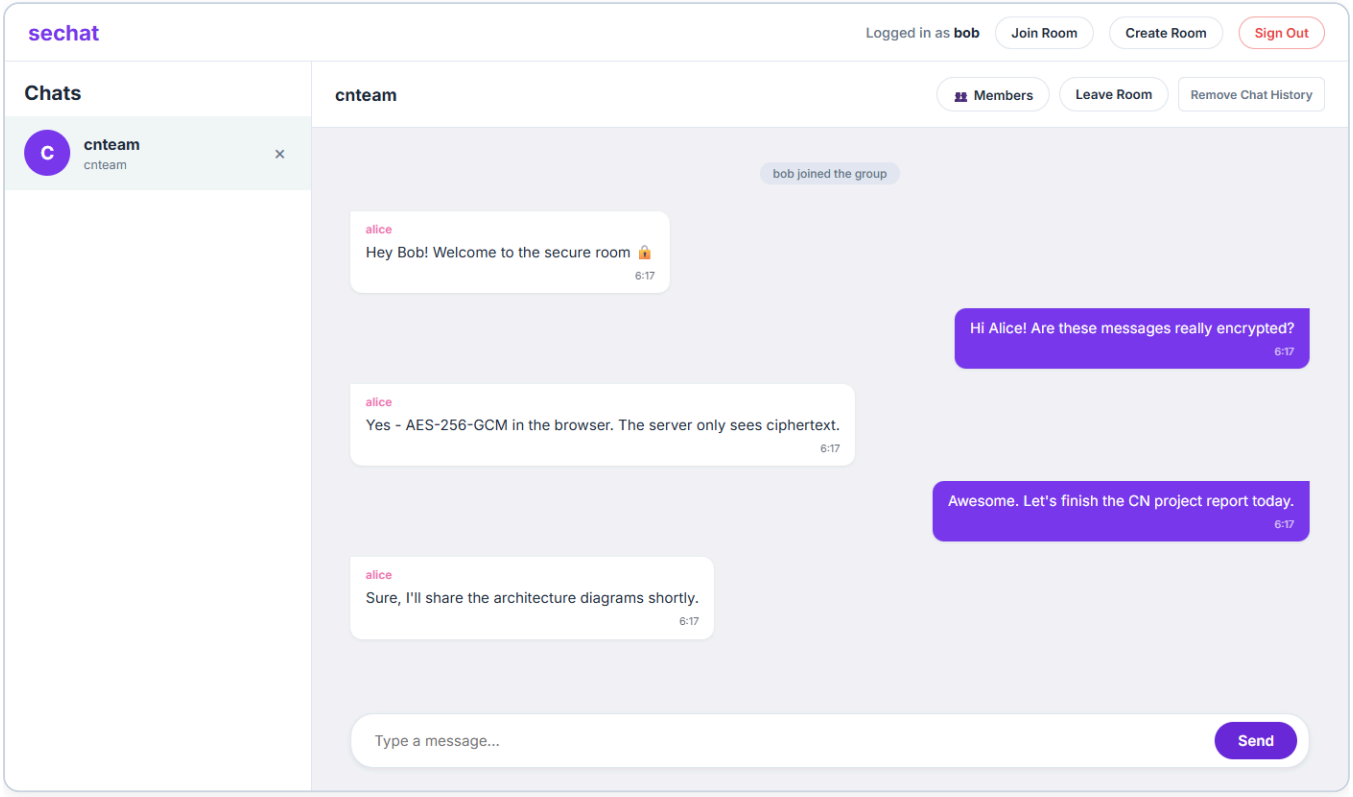


Figure 7 – Same conversation from Bob's view, with join notification

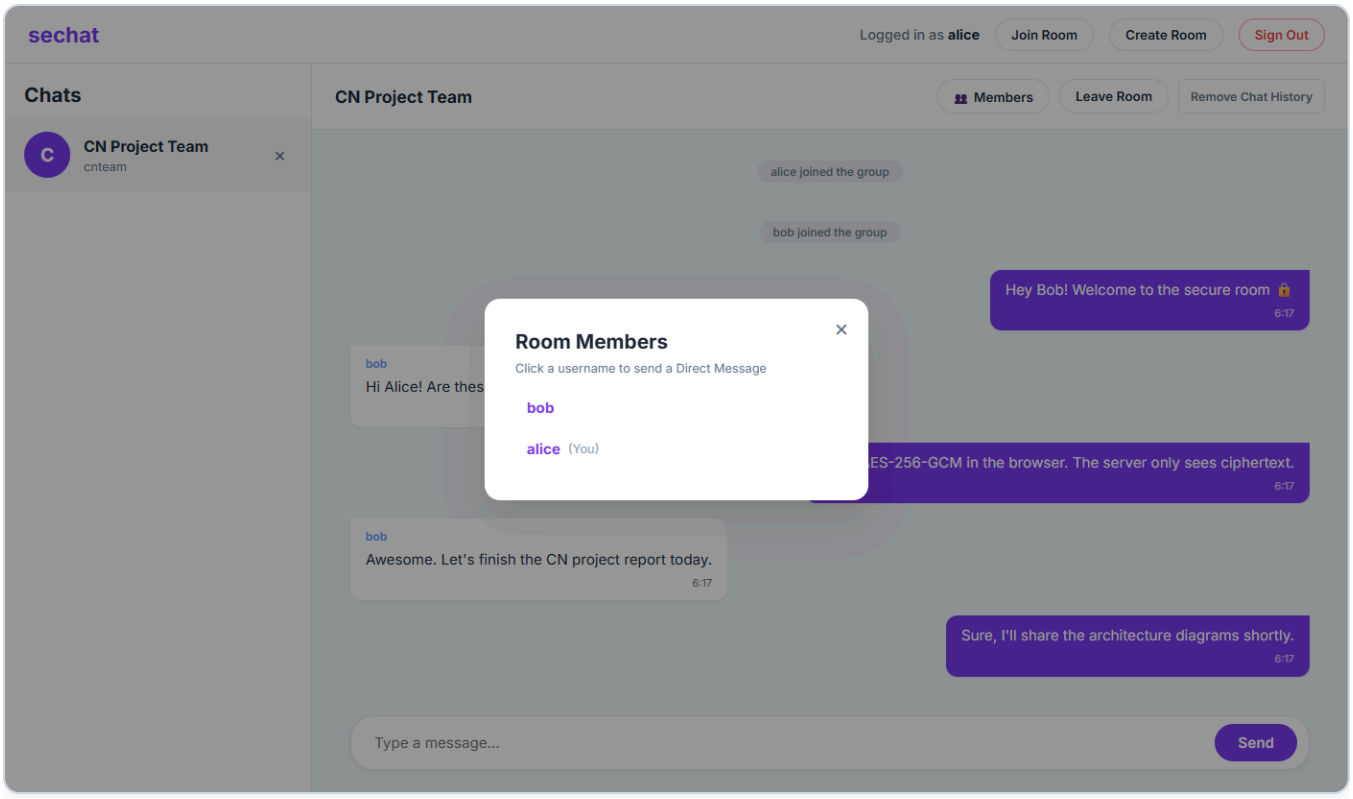


Figure 8 – Online members panel; click to start a DM

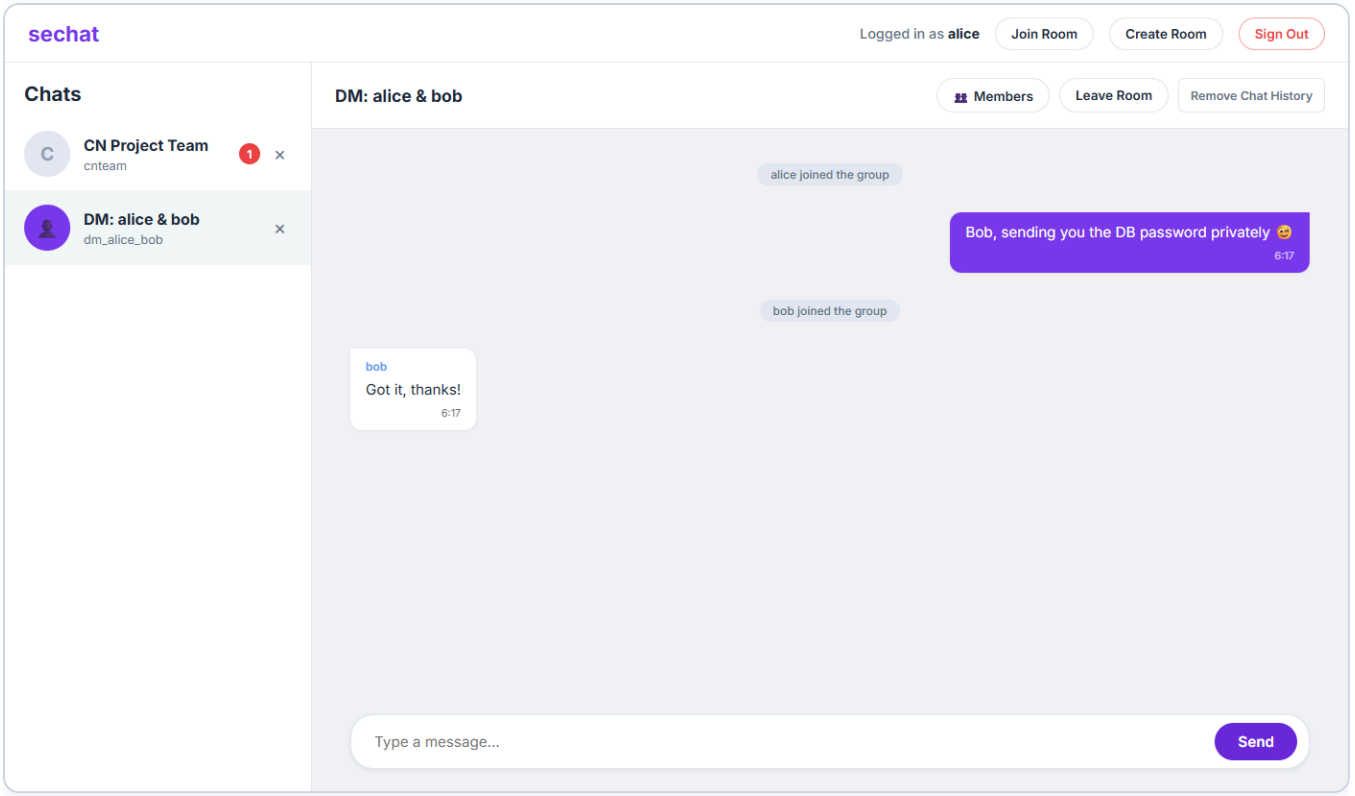
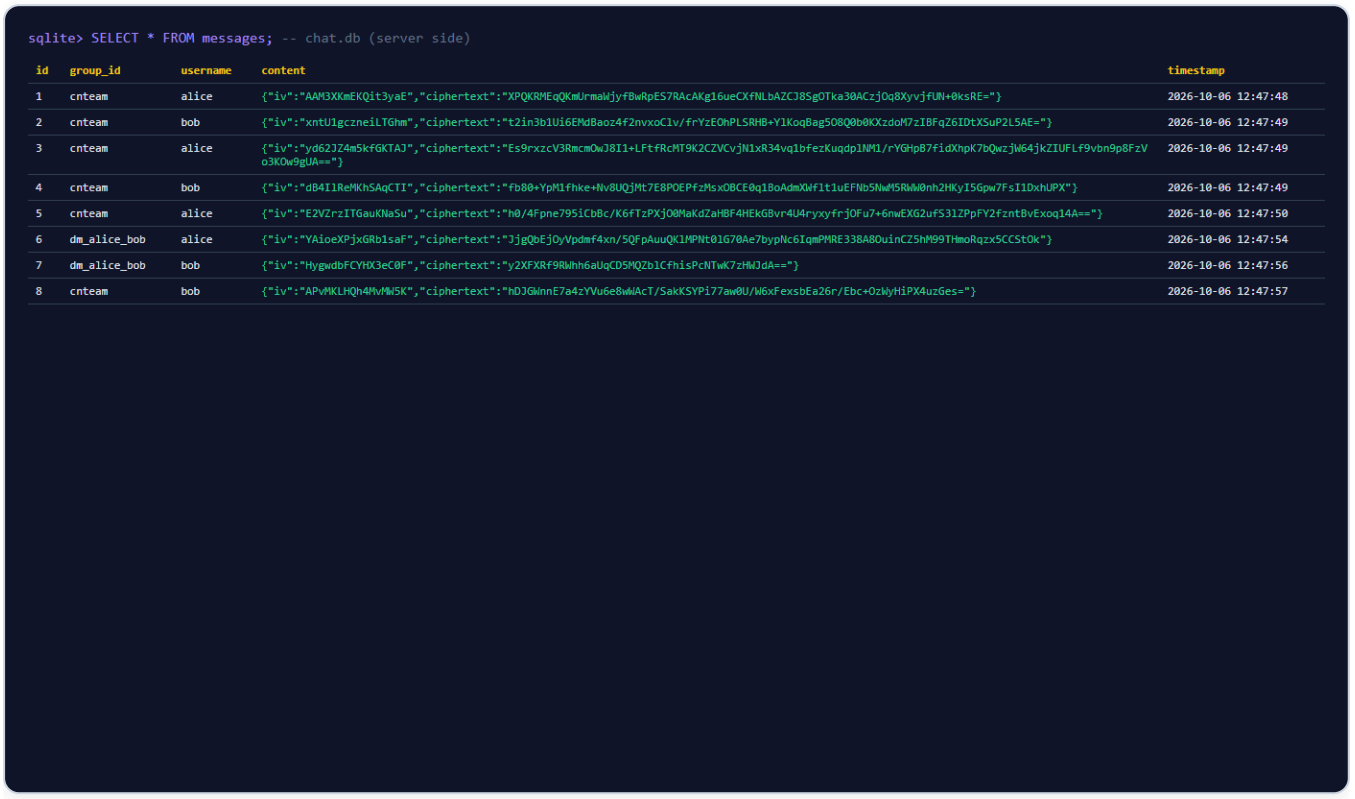


Figure 9 – Private direct message with unread badge on the group room



```
sqlite> SELECT * FROM messages; -- chat.db (server side)
```

id	group_id	username	content	timestamp
1	cnteam	alice	{"iv":"AAM3XKmEKQit3yaE","ciphertext":"XPQKRMEqQKmUrmaWjyfBwRpES7RAcAKg16ueCXfNLbAZCJ8SgOTka30ACzjOq8XyvjfUN+0ksRE="}	2026-10-06 12:47:48
2	cnteam	bob	{"iv":"xntU1gczneiLTGhm","ciphertext":"t2in3b1Ui6EMdBaoz4f2nvxoClv/frYzEOhPLSRHB+Y1KoqBag5O8Q0b0KXzdoM7zIBFqZ6IDtXSuP2L5AE="}	2026-10-06 12:47:49
3	cnteam	alice	{"iv":"yd62JZ4m5kfGKTAJ","ciphertext":"Es9rxzcV3RmcmOwJ8I1+LFtfRcMT9K2CZVCvjN1xR34vq1bfezKuqdplNM1/rYGHpB7fidXhpK7bQwzjW64jkZIUFLf9vbn9p8FzVo3KOw9gUA=="}	2026-10-06 12:47:49
4	cnteam	bob	{"iv":"dB4IlReMKhSAqCTI","ciphertext":"fb80+YpM1fhke+Nv8UQjMt7E8POEPfzMsxOBCE0q1BoAdmXWflt1uEFNb5NwM5RWW0nh2HKyI5Gpw7FsI1DxhUPX"}	2026-10-06 12:47:49
5	cnteam	alice	{"iv":"E2VZrzITGauKNaSu","ciphertext":"h0/4Fpne795iCbBc/K6fTzPXjO0MaKdZaHBF4HEkGBvr4U4ryxyfrjOFu7+6nwEXG2ufS3lZPpFY2fzntBvExoq14A=="}	2026-10-06 12:47:50
6	dm_alice_bob	alice	{"iv":"YAioeXPjxGRb1saF","ciphertext":"JjgQbEjOyVpdmf4xn/5QFpAuuQK1MPNt01G70Ae7bypNc6IqmPMRE338A8OuinCZ5hM99THmoRqzx5CCStOk"}	2026-10-06 12:47:54
7	dm_alice_bob	bob	{"iv":"HygwdbFCYHX3eC0F","ciphertext":"y2XFXRf9RWhh6aUqCD5MQZb1CfhisPcNTwK7zHWJdA=="}	2026-10-06 12:47:56
8	cnteam	bob	{"iv":"APvMKLHQh4MvMW5K","ciphertext":"hDJGWnnE7a4zYVu6e8wWAcT/SakKSYPi77aw0U/W6xFexsbEa26r/Ebc+OzWyHiPX4uzGes="}	2026-10-06 12:47:57

Figure 10 – What the server stores: only ciphertext in the messages table

12. Conclusion & Future Scope

sechat shows that a secure, real-time chat system can be built with a small, understandable codebase. WebSockets provide low-latency full-duplex communication, while client-side AES-256-GCM encryption makes the server a zero-knowledge relay. bcrypt hashing and one-time socket tokens protect accounts and rooms.

Future Enhancements

- Asymmetric key exchange (ECDH / Signal protocol) for DMs and forward secrecy
- TLS (wss:// / https://) deployment and JWT-based session authentication
- File and image sharing with encrypted blobs; typing indicators and read receipts
- Horizontal scaling with Redis pub/sub and PostgreSQL
- Mobile-responsive PWA with push notifications